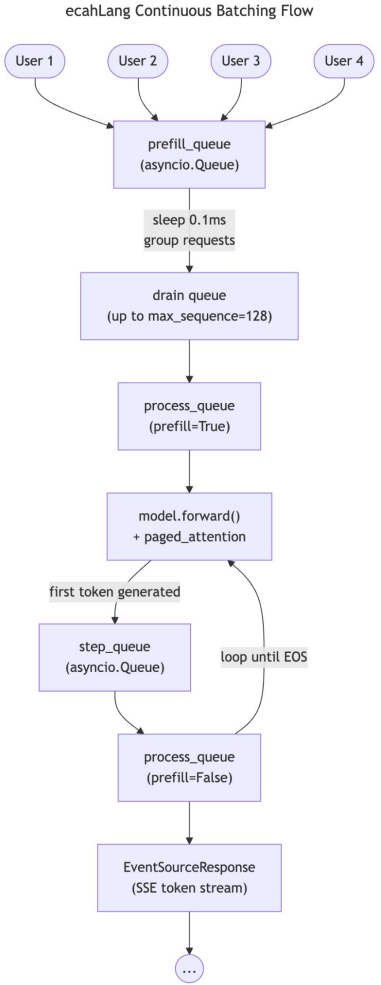
EcahLang

A lightweight llm inference & serving

Why EcahLang

Continuous Batching

Requests arrive at any time. ecahLang uses two async queues to handle them continuously

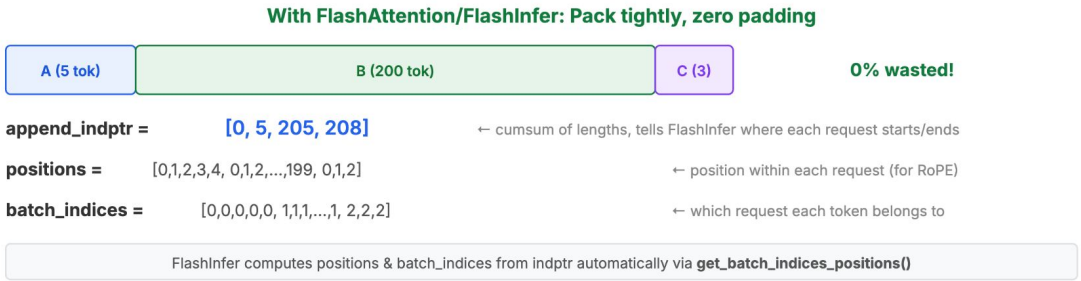


Continuous Batching - FlashAttention

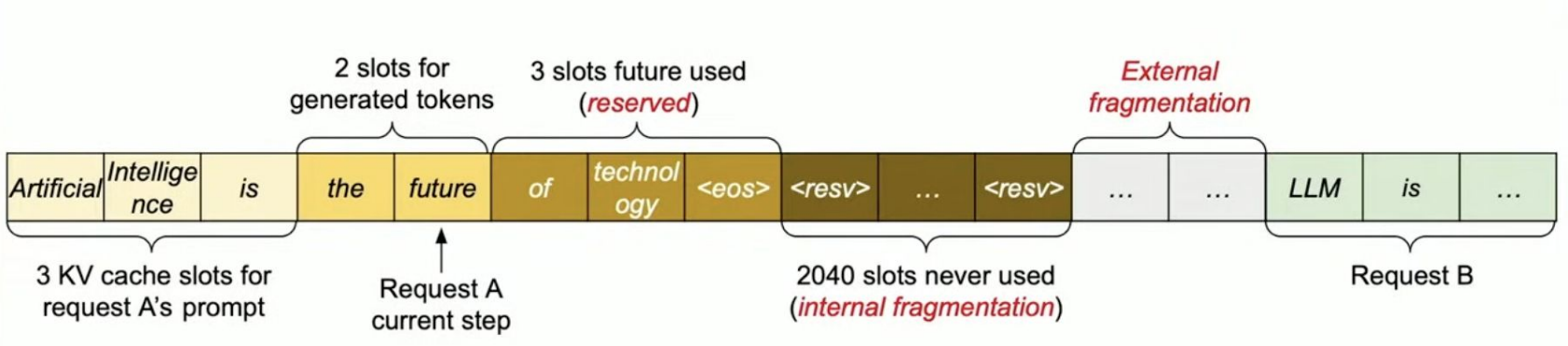
Continuous batching means different requests with different lengths in the same batch.
Traditional attention requires padding to the longest sequence. FlashAttention variable-length support solves this.

The Problem: Padding Wastes Compute

The Solution: Ragged Tensors via `indptr`



KV Cache problem

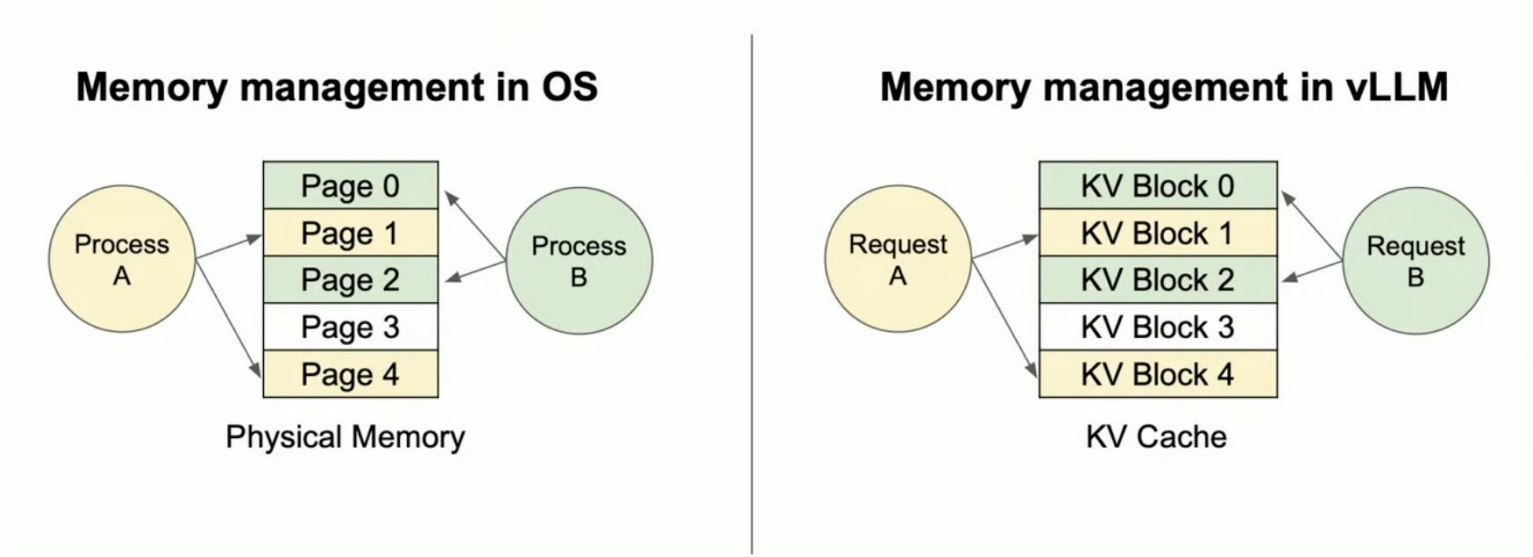


- **Internal Fragmentation**: over allocated due to unknown length
- **Reservation**: not used at the current step, but used in the future
- **External Fragmentation**: due to different sequence length

Only **20-40%** of KV Cache is utilized to store token states

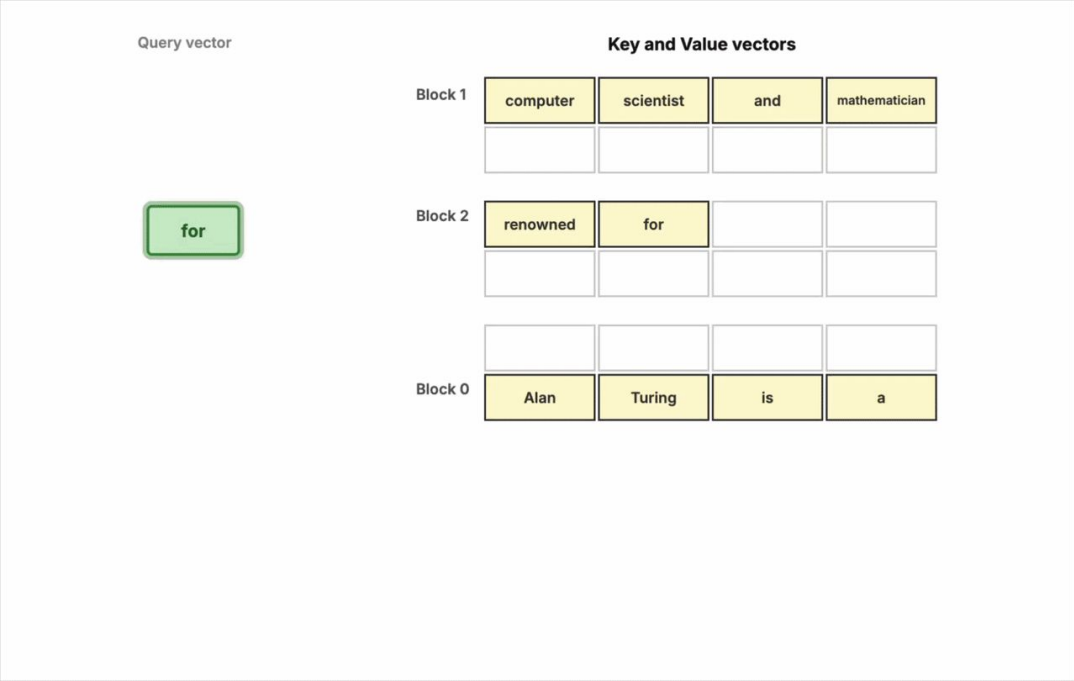
Paged KV Cache

Inspired by **virtual memory** and **paging**



PagedAttention

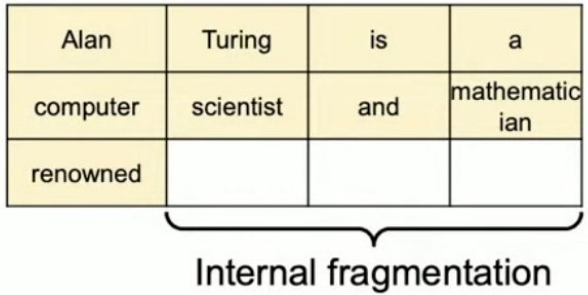
- An attention algorithm that allows for storing continuous keys and values in non-contiguous memory space



Logical & Physical Blocks

Paged KV Cache

- Minimal internal fragmentation
  - Only happens at the last block of a sequence
  - Bounded by the block size
- No external fragmentation



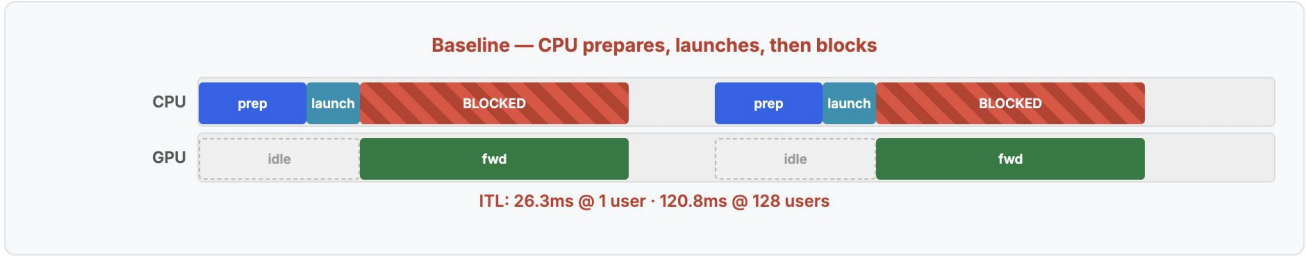
On average, the wasted space is less than **4%** of the kv cache space
(**3-5x** improved memory util)

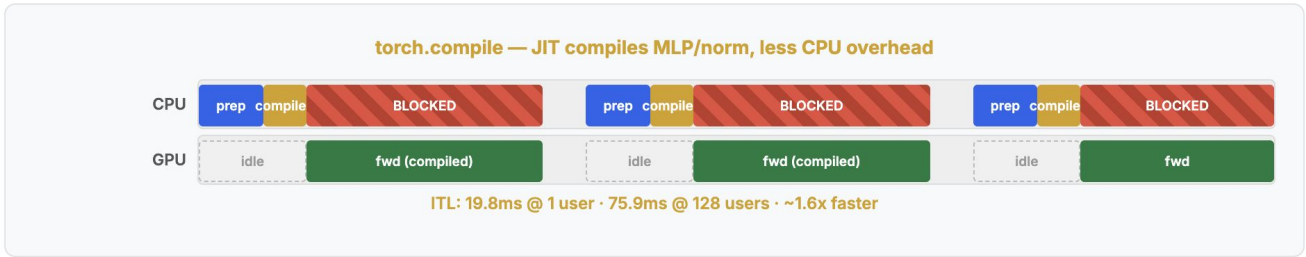
KV Cache - FlashInfer

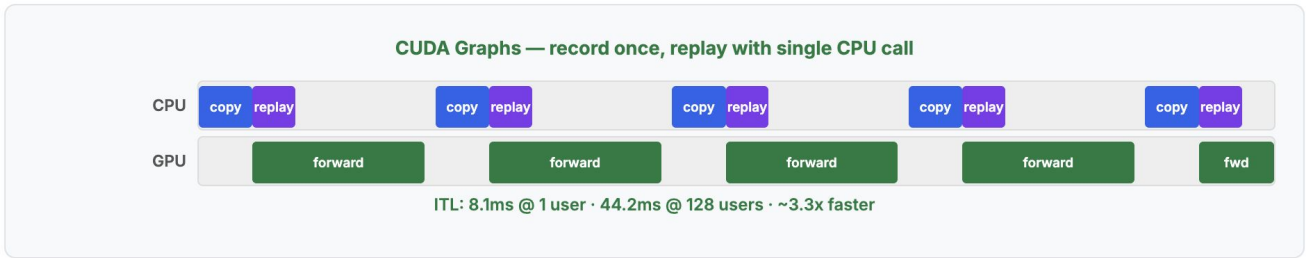
PagedAttention Kernel BatchPrefill & BatchDecode with paged KV cache	**KV Cache Management** append_paged_kv_cache() - scatter K,V into blocks
Fused Sampling (1 kernel, not 4) top_k_top_p_sampling_from_logits() - one cuda kernel	**Variable-Length Batching** Ragged tensors via indptr - no padding waste
Batch Planning wrapper.plan() - pre-compute metadata once per batch	**IO-Aware Tiled Attention** FlashAttention-style

ecahLang's attention function is ~20 lines because FlashInfer does all the heavy lifting.

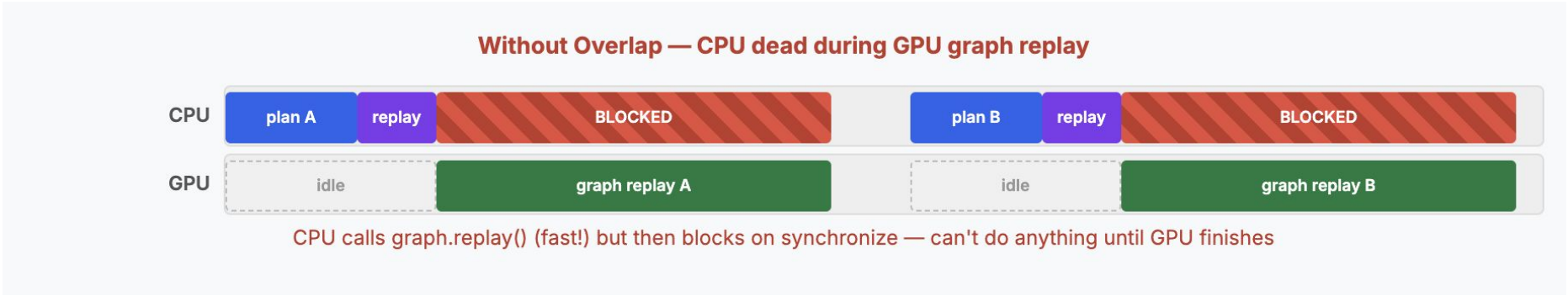
Reducing cpu overhead: CUDA Graphs







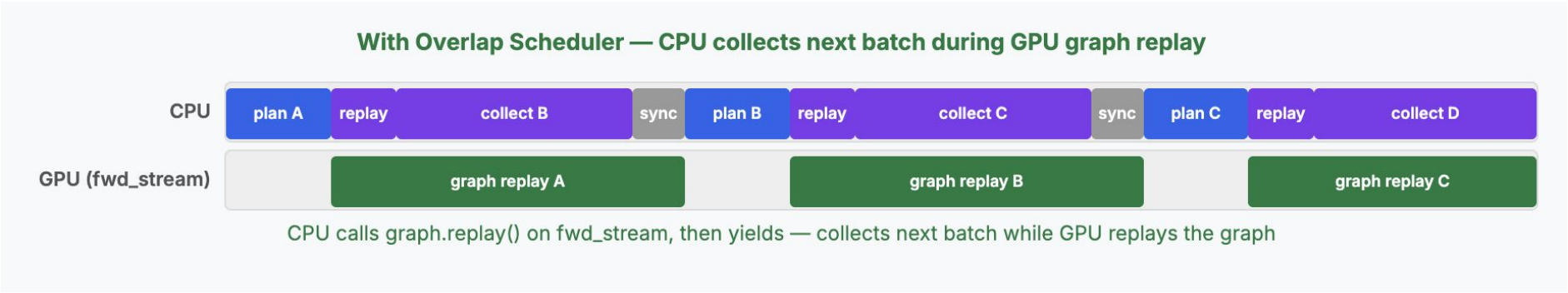
Overlap Scheduler



Without Overlap

`graph.replay()` launches all kernels (fast)
Then `synchronize()` blocks CPU
CPU sits idle during entire GPU replay
Next batch prep starts only after GPU done

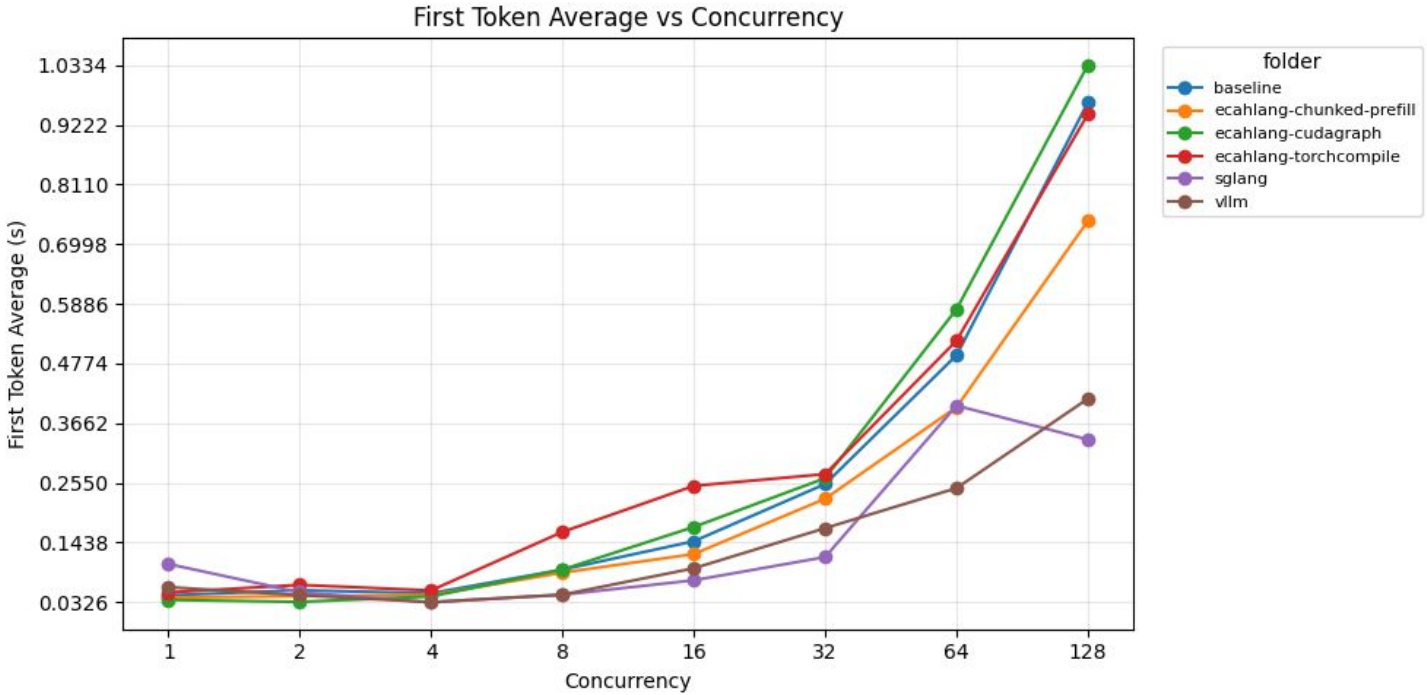
Overlap Scheduler



With Overlap

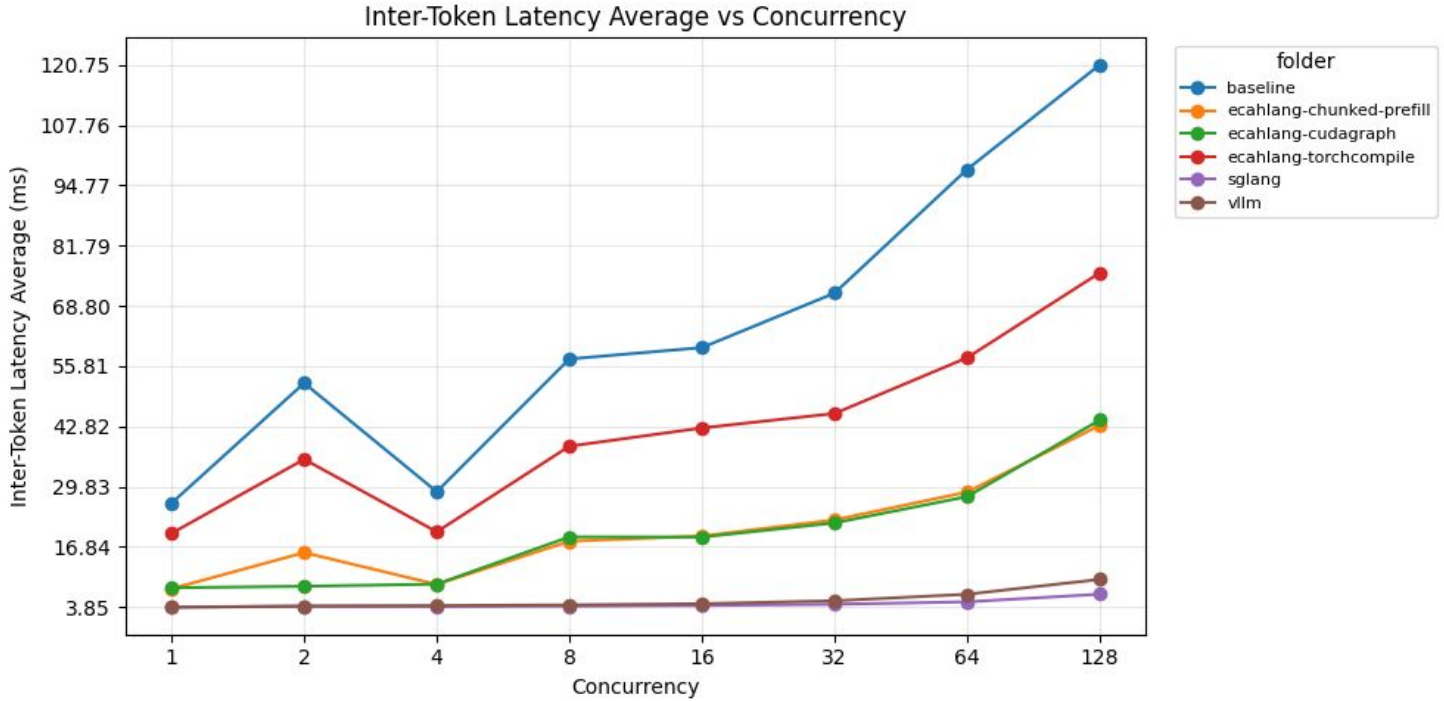
`graph.replay()` on `fwd_stream` (non-blocking)
`await asyncio.sleep(0)` yields to event loop
CPU drains queue, collects next batch
`fwd_stream.synchronize()` — next batch already ready

Benchmarks - Prefill



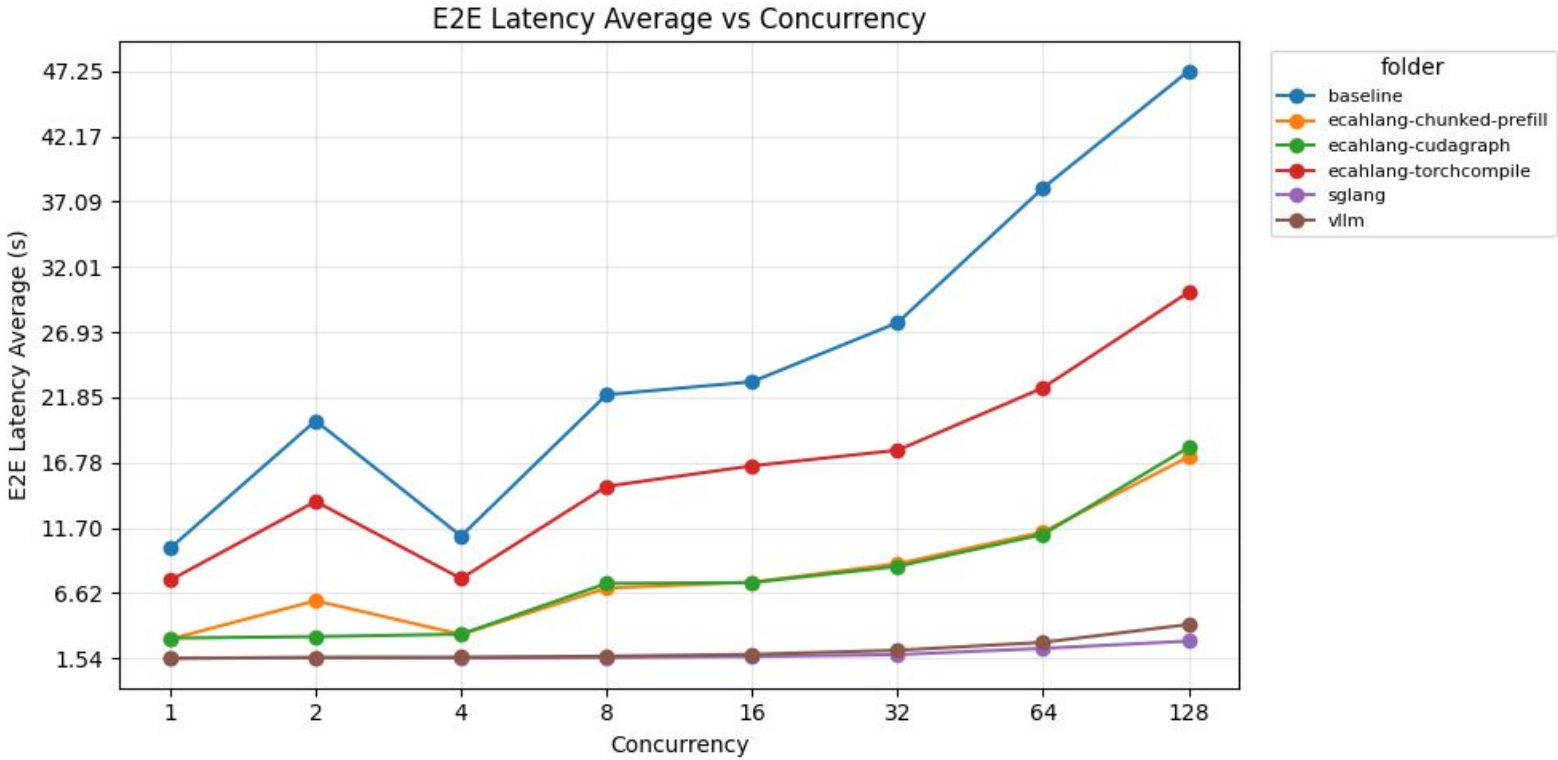
Chunked prefill has significantly improved TTFT, especially under high concurrency, reducing first token latency from **~1s** to **~0.77s** at 128 users.

Benchmarks - Decode



CUDA Graphs gave us roughly a 3x improvement in inter-token latency - dropping from **~120ms** to **~42ms** at 128 users.

Benchmarks - In-flight requests



Future outlook

1. Prefix Caching
2. LMCache
3. KV Cache offloading
4. Parallelism
5. Prefill-decode dissaggregation
6. Speculative decoding
7. Quantization

Source Code

README

ecahLang

Lightweight continuous batching inference engine for HuggingFace CausalLM models, built on FlashInfer. The name "ecah" is inspired by **Aisyah**, a lovely and popular Malaysian girl's name, a short and friendly nickname.

Features

- Continuous batching with paged KV cache
- FlashInfer paged prefill and decode attention
- CUDA Graph decode
- torch.compile decode
- CUDA stream overlap schedule
- Pre-allocated pinned sampling buffers
- Chunked prefill

Pre-requisites

```
curl -LsSf https://astral.sh/uv/install.sh | sh

uv venv --python 3.12
source .venv/bin/activate
```

Installation

```
pip install git+https://github.com/Scicom-AI-Enterprise-Organization/ecahLang
```

URL: https://github.com/Scicom-AI-Enterprise-Organization/ecahLang

Thanks